



Kaiju Finance

Security Review by Deadmanwalking

January 2025

Contents

1	Introduction	2
1.1	Disclaimer	2
1.2	About Kaiju Finance	2
2	Risk Classification	3
2.1	Impact	3
2.2	Likelihood	3
2.3	Action required for severity levels	3
3	Security Assessment Summary	4
3.1	Scope	4
4	Findings	5
4.1	[H-01] FixedCreditCostModel uses wrong precision which overcharges credit buyers	5
4.2	[M-01] Sandwiching addition and removal of supply can keep utilization artificially low and disable LIM	7
4.3	[M-02] No incentive to liquidate small positions and dust positions can avoid bad debt socialization	9
4.4	[M-03] Unbounded loops can cause out-of-gas and break agreement usability for short cycle periods	11
4.5	[M-04] Repayments get paused along with the whole protocol which can unfairly liquidate users after agreement expiration	13
4.6	[M-05] Liquidations of unhealthy borrowers are unable to function before the end of the grace period which increases likelihood of bad debt	14
5	Low Severity Findings	16
5.1	[L-01] Consider using Ownable2Step for contract ownership	16
5.2	[L-02] Missing sanity checks when creating agreements	16
5.3	[L-03] LIM suppliers are disproportionately exposed to bad debt during liquidations	17
5.4	[L-04] Agreements will not work with dynamic rate models	17
6	Informational Findings	19
6.1	[I-01] Withdraw amount can revert after bad-debt socialization due to missing shares to assets helper	19
6.2	[I-02] Using only one operator for user borrowing can create a bottleneck for high volume borrows in the same block	19

1 Introduction

This review was conducted by **deadmanwalking** as part of a private audit engagement with Kaiju finance, focusing on the protocol's core smart contracts.

Check out previous work [here](#) or reach out on Twitter **Oxdmanwalking**

1.1 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where I try to find as many vulnerabilities as possible. I cannot guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

1.2 About Kaiju Finance

Kaiju Finance is an interest-free revolving credit protocol implementing a statement-based cycle model. The smart contract system manages collateral-backed credit lines where on-time repayment incurs zero interest, while late payments trigger refinancing fees.

The protocol uses a monolithic contract design (`Kaiju.sol`) optimized for L2 deployment, with an agreement-based multi-market system where each agreement represents an isolated, ring-fenced credit market with unique asset pairs, risk parameters, and liquidity pools. The architecture ensures capital segregation—user collateral remains locked as safety buffers and is never reused as settlement liquidity. Payment provider integration is enabled through an operator system that allows linked borrowing on behalf of users, enabling seamless offchain spend flows backed by onchain debt tracking.

2 Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2.1 Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users.
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users.

2.2 Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

2.3 Action required for severity levels

- **Critical** - Must fix as soon as possible (if already deployed)
- **High** - Must fix (before deployment if not already deployed)
- **Medium** - Should fix
- **Low** - Could fix

3 Security Assessment Summary

Review commit hash:

- `ada0b3cd15613087fb1e55955a3a43790b0be779`

3.1 Scope

The following smart contracts were in scope of the audit:

- | | |
|---------------------------------|---|
| • <code>Kaiju.sol</code> | • <code>ParamsLib.sol</code> |
| • <code>Invariants.sol</code> | • <code>SafeTransferLib.sol</code> |
| • <code>AgreementLib.sol</code> | • <code>SharesMathLib.sol</code> |
| • <code>CycleLib.sol</code> | • <code>UserLib.sol</code> |
| • <code>ErrorsLib.sol</code> | • <code>FixedCreditCostModel.sol</code> |
| • <code>EventsLib.sol</code> | • <code>FixedLimRateModel.sol</code> |
| • <code>MathLib.sol</code> | • <code>FixedYieldRateModel.sol</code> |

4 Findings

4.1 [H-01] FixedCreditCostModel uses wrong precision which over-charges credit buyers

Severity: High

Impact: High Likelihood: High

Description

FixedCreditCostModel computes credit cost using a constant scaled to WAD ($1e18$) while the input `amount` is denominated in the loan token's native decimals (e.g., 6 for USDC). This causes the returned cost to be orders of magnitude larger than intended, leading users (or credit funders) to overpay when `Kaiju._addCredit()` charges and transfers cost in `ap.loanAsset`.

In FixedCreditCostModel, `COST_PER_DOLLAR_PER_SECOND` is defined using WAD scaling, assuming the amount is in 18 decimals:

```

1 contract FixedCreditCostModel is ICreditCostModel {
2     using MathLib for uint256;
3
4     address public immutable KAIJU;
5 >uint256 public constant COST_PER_DOLLAR_PER_SECOND = WAD * 0.04e6 / 365 days;
6
7     function cost(address funder, address user, uint256 amount, uint256 duration
8     )
9         external returns (uint256) {
10         require(msg.sender == KAIJU, "only Kaiju");
11
12         if (funderWhitelistEnabled) {
13             funderWhitelist[funder] -= amount;
14         }
15         if (userWhitelistEnabled) {
16             userWhitelist[user] -= amount;
17         }
18 >return (amount * duration).wMulUp(COST_PER_DOLLAR_PER_SECOND);
19     }
20 }
```

The `wMulUp(x, y)` function performs WAD-based multiplication (divide by $1e18$). This assumes `amount` is expressed with 18 decimals ("dollars" in WAD), but `amount` in Kaiju is in `ap.loanAsset` base units. For a 6-decimal stablecoin loan asset, `amount` is $1e6$ per "1 coin", not $1e18$. The model therefore mixes units: a WAD-scaled rate multiplied by a non-WAD principal.

In `Kaiju._addCredit()`, the cost is charged directly:

```

1 function _addCredit(AgreementId id, address user, uint256 amount, bool
2     startCycle)
3     internal returns (uint256 cost) {
4         AgreementParams storage ap = agreementParams[id];
5
6         uint256 duration = CycleLib.cyclesRemaining(ap.expiration, ap.cyclePeriod)
7         * ap.cyclePeriod;
8 >cost = ICreditCostModel(ap.creditCostModel).cost(msg.sender, user, amount, duration);
9
10        if (cost > 0) {
```

```
10 >IERC20(ap.loanAsset).safeTransferFrom(msg.sender, address(this), cost);  
11     agreementData[id].reserves += cost;  
12 }  
13 }
```

Example (USDC with 6 decimals):

- Credit amount: `amount = 100e6` (100 USDC in base units)
- Duration: `duration = 2,629,746` (1 month in seconds)
- Intended annual rate: \$0.04 per \$1 per year 4%/year
- Expected cost: ~ 0.33 USDC (`0.33e6`)
- Actual cost: **$\sim 333,554$ USDC (`333,554e6`) - 1 million times larger**

Recommendations

Consider having the cost per second be defined in the `FixedCreditCostModel` constructor and changed from the owner via a setter. Alternatively, to fix precision and keep the rate a constant, store the loan asset decimals on initialization and scale the value down by that factor.

Response

Fixed in [7afb7e9368ac1a006779564b03374566b6d79898](#).

4.2 [M-01] Sandwiching addition and removal of supply can keep utilization artificially low and disable LIM

Severity: Medium

Impact: Medium Likelihood: Medium

Description

Same-block supply add + supply remove can be executed with near-zero economic cost when `AgreementParams.withdrawLossDuration == 0`, enabling an actor to (a) temporarily consume `agreementData[id].supplyCap` and revert other LP deposits, and (b) manipulate the utilization inputs used by `refreshLim()` (which is manually called) by ensuring their temporary supply is present or absent around the privileged `refreshLim()` call.

In `_removeSupply()`, early-withdraw `lostReward` calculation includes two components:

```
1 function _removeSupply(...) internal {
2     ...
3     if (block.timestamp < ap.expiration) {
4 >lostReward = (ap.expiration - block.timestamp).wMulUp(au.rewardRate).wMulUp(removeRatio);
5 >lostReward += ap.withdrawLossDuration.wMulUp(au.rewardRate).wMulUp(removeRatio);
6         lostReward = MathLib.min(lostReward, amount);
7     }
8 }
```

If a user adds supply and removes supply in the same block, the remaining-time clawback can be minimized, and if `ap.withdrawLossDuration == 0` the extra penalty term is zero. Under the stated assumptions (fixed yield rate, `withdrawLossDuration = 0`), a same-block round-trip can be performed at ~ 0 penalty (aside from possible rounding dust and gas).

`refreshLim()` is manually invoked and uses instantaneous state to determine LIM availability:

```
1 function refreshLim(AgreementId id) external whenNotPaused nonReentrant {
2     ...
3 >uint256 targetSupply = agreementData[id].principal.wDivDown(ap.limUtilizationRatio);
4 >uint256 eligibleSupply = _availableSupply(id) + lastAvailability;
5
6     if (eligibleSupply < targetSupply) {
7 >ad.limAvailability = lastAvailability + (targetSupply - eligibleSupply);
8     } else if (_availableSupply(id) > targetSupply) {
9 >ad.limAvailability = 0;
10    }
11 }
```

An attacker can time add/remove supply to alter `_availableSupply(id)` at the moment `refreshLim()` executes, influencing whether LIM is enabled/disabled (i.e., whether `ad.limAvailability` is increased, set to 0, or left unchanged).

Even though the protocol is designed to be deployed on a chain with no mempool, the predictability of the LIM call means that a malicious actor can keep sending transactions around the time of high utilization with the hopes of influencing the outcome even if the `refreshLim` call does not happen in the same block.

Recommendations

The fix for this issue is non-trivial but the attack is also non-deterministic in chains that use a sequencer or with very small block times. Ideally, LIM supply would not be instantaneous and rely on an EMA. The simplest fix is to add a minimal `withdrawLossDuration`.

Response

Acknowledged. Additionally, this is not an issue on chains with sequencers.

4.3 [M-02] No incentive to liquidate small positions and dust positions can avoid bad debt socialization

Severity: Medium

Impact: Medium Likelihood: Medium

Description

Kaiju liquidations can be executed in small increments (via `repaidCycles` / `repaidAmounts`), while bad debt accounting is only triggered when the borrower's remaining collateral becomes exactly 0. This makes it possible (and often economically rational) for liquidators to leave "dust" collateral and/or "dust" debt, creating positions that are uneconomical to finish liquidating and delaying or preventing bad-debt socialization.

The liquidation flow:

- Computes `repaidAmount` as the sum of principal + interest actually covered inside `_repay(..., liquidation = true)`.
- Computes `seizedAmount` based on `repaidAmount` and caps it to the borrower's collateral with `MathLib.min(..., au.collateral)`.
- Reduces collateral by `seizedAmount`.
- Only if the borrower's collateral becomes 0, the contract computes and socializes remaining bad debt by reducing `ad.principal`, `ad.supply`, and `ad.limSupply`, and zeroing out `au.principals[cycle]`.

Because a liquidator controls `repaidCycles` / `repaidAmounts`, they can repay only a small amount and seize only a proportional amount of collateral, leaving:

- A small remaining collateral balance (`au.collateral > 0`) so the `if (au.collateral == 0)` branch is not executed, and/or
- A small remaining debt that is not socialized.

Separately, if the remaining debt/collateral becomes very small, the gas cost to liquidate can exceed the liquidation incentive, creating no incentive to liquidate small positions and causing such dust positions to persist and become permanent bad debt.

Bad debt (or near-bad-debt) can remain unsocialized for extended periods if liquidations are performed in a way that leaves `au.collateral > 0`. Liquidators may rationally avoid finishing liquidation of small residual positions when the remaining profit is below transaction cost, allowing dust positions to accumulate. Over time, the system can end up with a tail of unliquidatable accounts, creating permanent bad debt.

Recommendations

Consider enforcing minimum position sizes in 2 ways:

- Minimum borrow amount to prevent unprofitable to liquidate positions (can be small for small purchases)
- Enforce that if liquidation would leave the borrower with remaining collateral (or remaining debt) below a configured threshold, require the liquidation to fully close the position (e.g., force `au.collateral` to 0 and close all remaining debt) or revert.

Response

Acknowledged.

4.4 [M-03] Unbounded loops can cause out-of-gas and break agreement usability for short cycle periods

Severity: Medium

Impact: High Likelihood: Low

Description

Multiple user-facing (and liquidation-critical) iterations iterate from cycle 0 to the current cycle number. If `ap.cyclePeriod` is small and/or the agreement expiration is really far in the future, `CycleLib.cycleNumber(...)` becomes large, making these loops unbounded in practice and potentially reverting with out-of-gas. This can block health checks, borrowing, collateral removal, repayments, and liquidation flows that depend on these computations, effectively DOSing the entire agreement.

The `UserLib` functions loop from cycle 0 to current cycle without bounds:

```

1 function userDebt(AgreementUser storage au, AgreementParams storage ap)
2     internal
3     view
4     returns (uint256 principal, uint256 interest)
5 {
6     >for (
7     >uint256 cycle = 0;
8     >cycle <= CycleLib.cycleNumber(au.creditStartTime, block.timestamp, ap.cyclePeriod);
9     >cycle++
10    >) {
11         principal += au.principals[cycle];
12         interest += interestFromCycle(au, ap, cycle, au.principals[cycle]);
13     }
14 }
15
16 function userDebtCycles(AgreementUser storage au, AgreementParams storage ap)
17     internal
18     view
19     returns (uint256[] memory principals, uint256[] memory interests)
20 {
21     >uint256 currentCycle = CycleLib.cycleNumber(au.creditStartTime, block.timestamp,
22         ap.cyclePeriod);
23     >principals = new uint256[](currentCycle + 1);
24     >interests = new uint256[](currentCycle + 1);
25     for (uint256 cycle = 0; cycle <= currentCycle; cycle++) {
26         principals[cycle] = au.principals[cycle];
27         interests[cycle] = interestFromCycle(au, ap, cycle, principals[cycle]);
28     }

```

These loops are not capped and scale linearly with the number of elapsed cycles since `au.creditStartTime`. Agreements with short `ap.cyclePeriod` can accumulate many cycles quickly.

If the number of elapsed cycles is large enough:

- View functions that compute debt/health may revert due to gas or memory constraints (notably `userDebtCycles()` allocating `currentCycle + 1` arrays).
- State-changing functions that depend on `au.isHealthyValue(ap)` or `au.userDebt(ap)` can become unusable, since they transitively call `userDebt(...)`.

- Examples include `_borrow()` (calls `au.isHealthyValue(ap)`), `_removeCollateral()` (calls `au.isHealthyValue(ap)`), and `liquidate()` (calls `au.isHealthyValue(ap, collateralPrice)`).
- This can effectively DOS an agreement for users with long-lived positions and/or short cycle configurations, preventing normal position management and resolution.

Recommendations

Consider enforcing a minimum `cyclePeriod` (or minimum agreement duration / maximum cycles) when enabling an agreement, so `CycleLib.cycleNumber(...)` is upper-bounded over the agreement lifetime.

Response

Acknowledged. This is not a concern for the agreement parameters expected to be used in production.

4.5 [M-04] Repayments get paused along with the whole protocol which can unfairly liquidate users after agreement expiration

Severity: Medium

Impact: Medium Likelihood: Medium

Description

When the protocol is paused via `pause()`, both `repay()` and `liquidate()` are blocked by `whenNotPaused`. During the pause, time-based liquidation eligibility continues to advance (e.g., `block.timestamp` can exceed `ap.expiration + ap.repaymentPeriod`). After `unpause()`, borrowers can become immediately liquidatable based on elapsed time, without having had an opportunity to repay while the protocol was paused.

`Kaiju.pause()` calls `_pause()` and blocks state-changing user actions guarded by `whenNotPaused`, including:

- `repay(AgreementId id, address user, uint256[] cycles, uint256[] amounts)`
- `liquidate(AgreementId id, address borrower, uint256[] repaidCycles, uint256[] repaidAmounts, bytes data)`

While paused, borrower debt and liquidation eligibility conditions that depend on `block.timestamp` can continue to mature. In `liquidate()`, liquidation can proceed even if the position is healthy once the agreement has passed `expiry + repayment period`:

- `require(!isHealthyValue || ap.expiration + ap.repaymentPeriod < block.timestamp, ErrorsLib.HEALTHY_POSITION);`

This creates a conflict: borrowers are unable to repay during the pause, but after unpausing they may be immediately liquidatable due to the agreement expiring.

Borrowers can lose collateral due to liquidations that become eligible solely because `block.timestamp` advanced during a protocol pause, when borrowers could not call `repay()` to reduce debt or close cycles.

Recommendations

Consider adding a post-unpause liquidation grace period, capped at the pause duration, so borrowers are not immediately liquidatable due to time elapsed during a pause. This could be done either via a privileged function or automatically by modifying the pause and unpause functions to keep track of pausing time.

Response

Fixed.

4.6 [M-05] Liquidations of unhealthy borrowers are unable to function before the end of the grace period which increases likelihood of bad debt

Severity: Medium

Impact: Medium Likelihood: Medium

Description

Liquidation flow requires the borrower's debt cycles to have passed the repayment grace period before `_repay(..., liquidation=true)` will process them. This prevents liquidation of unhealthy positions during an active cycle (or during the repayment window), delaying liquidation while collateral value may continue to fall.

The `_repay` function enforces a strict time requirement for liquidations:

```

1 function _repay(
2     AgreementId id,
3     address user,
4     uint256[] calldata cycles,
5     uint256[] calldata amounts,
6     bool liquidation
7 ) internal returns (uint256 sumPrincipal, uint256 sumInterest) {
8     ...
9     for (uint256 i = 0; i < cycles.length; i++) {
10         require(amounts[i] > 0, ErrorsLib.ZERO_AMOUNT);
11
12 > if (liquidation) {
13 > require(
14 > block.timestamp > CycleLib.cycleEnd(au.creditStartTime, cycles[i], ap.cyclePeriod)
15 > + ap.repaymentPeriod,
16 > ErrorsLib.CYCLE_NON_LIQUIDATABLE
17 > );
18 > }
19
20         (uint256 principalCovered, uint256 interestCovered) =
21             au.repayCoverage(ap, cycles[i], amounts[i]);
22         au.principals[cycles[i]] -= principalCovered;
23         ...
24     }
25 }

```

`Kaiju.liquidate(...)` allows liquidation when a position is unhealthy (`!isHealthyValue`) even if the current cycle has not ended. However, it then calls `_repay(..., liquidation=true)`, and `_repay` enforces that each `cycles[i]` provided is strictly past `cycleEnd + ap.repaymentPeriod` when `liquidation == true`.

As a result, liquidators cannot repay debt from the current cycle (or the most recent cycle still in the repayment window), even when the borrower is already undercollateralized. If the borrower's debt is mostly (or entirely) in the current cycle, liquidation cannot proceed until the cycle ends and the repayment period elapses.

This creates the following impacts:

- Underwater borrowers can remain unliquidatable until `cycleEnd + repaymentPeriod`, increasing the time window where collateral price may continue to drop.

- Increased probability of bad debt at liquidation time, since collateral may be insufficient by the time liquidation is permitted.
- Liquidity providers (regular supply and LIM supply) bear higher loss risk when bad debt is socialized (e.g., in `liquidate` when `au.collateral == 0` and the protocol reduces `ad.supply / ad.limSupply`).

Recommendations

Consider removing the check in `_repay` to allow liquidations to go through for unhealthy borrowers even before cycle expiration.

Response

Fixed in [5450d1bdd330c91d2cf750512eb6331f2d104167](#).

5 Low Severity Findings

5.1 [L-01] Consider using Ownable2Step for contract ownership

Severity: Low

The owner role in Kaiju can be transferred in a single step via `setOwner(address newOwner)`, which immediately updates `owner`. `setOwner` directly assigns `owner = newOwner` after checks. There is no pending owner / accept step which increases error surface.

If `newOwner` is set incorrectly (e.g., wrong address), control is immediately transferred and cannot be recovered via an onchain acceptance step. Operational mistakes can irreversibly impact admin actions gated by `onlyOwner`.

Use a two-step ownership transfer pattern (set `pendingOwner`, require `pendingOwner` to call `acceptOwner()`), and emit events for both steps. Consider using OpenZeppelin's `Ownable2Step` for this.

Response

Fixed using `AccessControlUpgradeable`.

5.2 [L-02] Missing sanity checks when creating agreements

Severity: Low

`enableAgreement()` accepts an `AgreementParams` struct and enables an agreement with minimal validation. As a result, misconfigured agreements can be created that can DOS borrowers, suppliers and cause various issues.

Agreements are created/enabled via `enableAgreement(AgreementParams ap, address creditor, uint256 initialReserves)`. The function does not sanity-check `AgreementParams` fields (e.g., asset addresses, oracle, rate models, time parameters).

If an agreement is enabled with invalid parameters, downstream user flows may revert unexpectedly (e.g., oracle calls), produce incorrect accounting (e.g., zero/incorrect periods), or become non-functional causing DOS of the agreement and its users.

Consider adding explicit sanity checks in `enableAgreement()`. At minimum, validate:

- `creditor != address(0)`
- `ap.loanAsset != address(0)` and `ap.collateralAsset != address(0)` and `ap.loanAsset != ap.collateralAsset`
- `ap.oracle != address(0)` and that `IOracle(ap.oracle).price()` does not revert (optionally in a try/catch)
- Model addresses: `ap.creditCostModel != address(0)`, `ap.yieldRateModel != address(0)`
- Time parameters: `ap.expiration > block.timestamp`, `ap.cyclePeriod > 0`
- Economic bounds: `ap.lltv <= WAD`, etc.

Given that the creation of agreements is privileged, the burden of verification is shifted to the owner.

Response

Fixed.

5.3 [L-03] LIM suppliers are disproportionately exposed to bad debt during liquidations

Severity: Low

LIM suppliers can be locked (via vesting) while regular suppliers can withdraw earlier (subject to `lostReward`). If a large liquidation with bad debt is imminent (e.g., collateral crash), regular suppliers can exit before the liquidation and avoid (or reduce) loss from bad-debt socialization, while LIM suppliers cannot exit until `vestTime`, concentrating the remaining loss on LIM.

Kaiju has two supply classes:

- Regular supply (`agreementData[id].supply`) with user shares `users[user].supplyShares`, withdrawable via `removeSupply()` at any time before expiration, subject to `lostReward`.
- LIM supply (`agreementData[id].limSupply`) with per-user vesting shares `users[user].limSupplyShares`, withdrawable only after `vestTime` via `removeLimSupply()`.

During a liquidation with bad debt, the system reduces `ad.supply` and `ad.limSupply` (bad debt is socialized across both). Because regular supply is withdrawable (with a known penalty), but LIM is time-locked, there is an incentive for regular suppliers to front-run an anticipated bad-debt socialization event and exit, leaving a larger fraction of the remaining loss to be borne by locked LIM suppliers.

Document the behavior and surface all relevant risks for LIM suppliers so that they are aware of both the opportunity cost of locking their funds and the potential bad debt issues.

Response

Acknowledged.

5.4 [L-04] Agreements will not work with dynamic rate models

Severity: Low

Kaiju's supply accounting assumes a single, effectively static yield rate per user position, but the implementation aggregates per-deposit reward entitlements into `agreementData[id].users[user].rewardR` and later applies early-withdraw penalties pro-rata using a single blended rate. If `IYieldRateModel(ap.yieldRa` is dynamic over time, partial withdrawals cannot be attributed to specific deposits, so the computed `lostReward` may not match the rewards that were actually pre-paid for the withdrawn portion.

When supplying, Kaiju calculates and pre-pays a fixed yield for the remaining time-to-expiration at the current yield rate. When removing supply early, Kaiju computes the lost reward from the same single aggregate rate. There is no per-deposit attribution, so with a dynamic yield model, `lostReward` is calculated as if the withdrawn shares represent a pro-rata

slice of a single blended `au.rewardRate`, not the actual deposited tranche(s) the withdrawal economically corresponds to.

Document the behavior of the yield rate model and when agreement creation becomes permissionless, enforce that only whitelisted models can be used (similar to Morpho's whitelisted components/parameters).

Response

Acknowledged.

6 Informational Findings

6.1 [I-01] Withdraw amount can revert after bad-debt socialization due to missing shares to assets helper

Severity: Informational

After bad-debt socialization reduces `agreementData[id].supply`, a supplier may be unable to withdraw the original notional `amount` they supplied because `removeSupply()` converts `amount` to shares using `toSharesUp(...)`, which can exceed the user's recorded `au.supplyShares` and revert. The protocol does not expose a view helper (or a shares-based withdrawal) to let users compute the maximum withdrawable `amount` for their current shares.

`_removeSupply()` converts a user-provided `amount` (assets) to `amountShares` and then checks it against the user's `au.supplyShares`. This requires callers to provide an `amount` that maps (via the current pool exchange rate) to no more shares than they own.

During liquidation bad-debt handling, the agreement can reduce `ad.supply` (and/or `ad.limSupply`) while leaving share balances intact. This changes the shares to assets exchange rate. After `ad.supply` decreases, converting the same notional `amount` to shares can require more shares than before, and `amount.toSharesUp(...)` can exceed `au.supplyShares`, causing `removeSupply()` to revert even if the user is trying to redeem "their full position" by nominal value.

There is no public/view function for users to compute assets equivalent of `au.supplyShares` at current `agreementData[id].supply / agreementData[id].supplyShares`.

Expose deterministic shares to assets conversion helpers for suppliers, using the same rounding conventions as the core logic just like ERC4626 to improve dev UX.

6.2 [I-02] Using only one operator for user borrowing can create a bottleneck for high volume borrows in the same block

Severity: Informational

Kaiju relies on a single `operator` address for `linkedBorrow()` execution. If the same keeper/relayer address must submit transactions for many users, the backend cannot safely parallelize submissions due to nonce ordering, which can introduce confirmation delays or reverts for concurrent card swipes.

`linkedBorrow()` is restricted by `onlyOperator`, which checks `msg.sender == operator`. This design implies a single authorized EOA/keeper (or a single key controlling an account) is used to submit linked borrow transactions. On EVM chains, transactions from the same sender are strictly ordered by nonce. When multiple swipes occur close together, subsequent transactions must wait for earlier nonces to be mined or replaced, creating a serialization bottleneck even if liquidity/collateral checks would otherwise pass.

This can cause increased end-to-end latency for `linkedBorrow()` when multiple users transact concurrently via the same keeper, causing potential card UX to degrade.

```
1 modifier onlyOperator() {
2     require(msg.sender == operator, ErrorsLib.UNAUTHORIZED);
3     _;
4 }
```

```
5
6 function linkedBorrow(AgreementId id, address user, address destination, uint256
   amount)
7     external onlyOperator {
8         require(agreementData[id].users[user].links[destination], ErrorsLib.
LINK_NOT_ENABLED);
9         _borrow(id, user, destination, amount);
10 }
```

Replace the single `operator` with a set of authorized operators (e.g., `mapping(address => bool) public isOperator;`) and allow multiple keepers/relayers to submit `linkedBorrow()` in parallel using distinct sender accounts (distinct nonces).